redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

Team Name :ALCATRAZ

Team Leader Name : Vaishnavi Singh

Problem Statement : Intelligent Candidate Discovery

What is your proposed solution?

Our proposed solution is an AI-powered candidate ranking engine that takes a raw job description and a pool of candidate profiles, and returns a precise, explainable, ranked shortlist.

The system works in two stages. First, it parses the JD into structured requirements using targeted extraction rather than treating the JD as a flat block of text. Every candidate is then scored against these requirements across five dimensions: semantic relevance (via sentence embeddings), skills match, education, career history, and behavioural signals. These combine into a weighted composite score, which also passes through a title-relevance check and an automated honeypot detector that flags and zeroes out suspicious or fabricated profiles.

The top-scoring candidates from this first pass are then re-ranked using a cross-encoder to sharpen precision at the top of the list without paying that cost across the entire candidate pool. The final output is a ranked shortlist with a short, evidence-based reasoning string for every candidate, generated directly from their real profile data (title, matched skills, notice period, response rate, GitHub activity), so every ranking decision is traceable and auditable rather than a black box.

What differentiates your approach from traditional candidate matching systems?

Unlike traditional keyword-based systems, our approach doesn't just count matching words. It first understands the meaning of the JD using semantic embeddings, then combines that with structured scoring across skills, education, career history, and behavioural signals followed by a cross-encoder re-ranking pass to sharpen precision on the top candidates. It also runs automated fraud detection to catch suspicious or fabricated profiles, and generates short, evidence-based reasoning for every ranked candidate, so results go beyond surface-level matching to reflect genuine fit and are fully explainable.

What are the key requirements extracted from the JD?

The system extracts four structured requirements from the raw job description: experience range, notice-period expectations, preferred locations, and required skills (matched against a curated list of relevant technical keywords like Python, LLMs, embeddings, and MLOps tools). These structured fields drive scoring downstream, instead of relying on the raw JD text alone.

How does your solution evaluate candidate fit beyond keyword matching?

Fit is evaluated across five weighted signals:

- semantic relevance (40%): sentence embeddings compare the meaning of the JD against a candidate's full profile
- Skills (20%): weighted by proficiency level, how long the skill was used, endorsements, and assessment scores.
- Career relevance (15%): checks whether a candidate's actual job history reflects real, hands-on work on the JD's core concepts.
- Education (15%): institution tier and field relevance.
- Behavioural/activity signals (10%): recency of activity, recruiter response rate, notice period, relocation willingness.

How does your system retrieve, score, and rank candidates?

Stage 1(Retrieval & Coarse Scoring): Every candidate profile is flattened into a single text blob (headline, summary, job titles, job descriptions, skills, education, certifications) and embedded using a bi-encoder (all-MiniLM-L6-v2). All candidate embeddings are computed in a single batched pass for efficiency. Cosine similarity between the JD embedding and each candidate embedding gives a fast semantic relevance score across the entire pool. This is combined with rule-based skills, education, career-relevance, and behavioural scores into a weighted composite score, adjusted by a title-relevance penalty and honeypot detection. Candidates are sorted by this composite score, and the top 500 move forward.

Stage 2 (Precision Reranking): The top 500 candidates are re-scored using a cross-encoder (cross-encoder/ms-marco-MiniLM-L-6-v2), which reads the JD and each candidate's text jointly rather than comparing separate embeddings giving a much more accurate relevance judgment, at a higher compute cost that's now bounded to just 500 candidates instead of the full pool. These scores are normalized and blended with the Stage 1 score (60% cross-encoder, 40% original composite) to produce the final ranking. **Final ranking**: Candidates are sorted by this blended score, and the top 100 are output with rank, score, and reasoning.

What models, algorithms, or heuristics are used?

Models

- **Bi-encoder (sentence-transformers/all-MiniLM-L6-v2):** embeds the JD and every candidate profile independently for fast semantic similarity scoring across the full candidate pool via cosine similarity.
- **Cross-encoder (cross-encoder/ms-marco-MiniLM-L-6-v2):** jointly scores JD-candidate pairs for high-precision reranking, applied only to the top 500 candidates from Stage 1 to keep compute cost bounded.

Algorithms:

- Cosine similarity between JD and candidate embeddings for semantic relevance.
- Weighted linear combination of five scoring dimensions (semantic, skills, education, career, behavioural) into a single composite score.
- Min-max normalization of cross-encoder scores before blending with Stage 1 scores.

Heuristics:

- Skills scoring: weighted by proficiency (expert/advanced/intermediate/beginner), skill duration, endorsement count, and assessment test scores.
- Education scoring: institution tier plus relevance of field of study to the JD domain.
- Career-relevance scoring: keyword/concept matching against a candidate's actual job descriptions (not just their skills list), rewarding real hands-on work and penalizing keyword-stuffed profiles with no supporting history.
- Behavioural scoring: activity recency, recruiter response rate, average response time, notice period fit, relocation willingness, GitHub activity (for technical roles), interview completion rate, offer acceptance rate, and recent recruiter/application engagement.
- Title-relevance penalty : a 0.5x multiplier applied when a candidate's current title shares no meaningful overlap with the JD.
- Honeypot/fraud detection: a flag-based heuristic (e.g., implausibly many "expert" skills, perfect scores across every engagement metric simultaneously, high application volume with near-zero response rate) that zeroes out suspicious profiles' final scores.

How are multiple candidate signals combined into a final ranking?

Stage 1 (Composite score): Final = (0.40 × semantic + 0.20 × skills + 0.15 × education + 0.10 × behavioural + 0.15 × career) × title_penalty

Stage 2 (Final blend):final = (0.60 × cross_encoder_score) + (0.40 × stage1_score)

Only the top 500 candidates from Stage 1 go through this step. The cross-encoder score is normalized (min-max) before blending, and it's weighted more heavily (60%) than the Stage 1 score (40%) because it's a more accurate, JD-aware relevance judgment while the Stage 1 score still anchors the result so a single cross-encoder quirk can't wildly override the broader signal profile.

How are ranking decisions explained?

Every ranked candidate gets a short, auto-generated reasoning string (generate_reasoning) that's built directly from their real profile data rather than free-text generation so the explanation is deterministic and traceable back to actual fields.
For each candidate, the reasoning includes: current title, years of experience, their top matched skills (specifically the ones that overlap with the JD's required skills, not just any skills they've listed), notice period, recruiter response rate, and GitHub activity score. It's assembled as a single readable line.

If a candidate was flagged by the honeypot detector, [HONEYPOT DETECTED] is prepended to their reasoning in the output, so suspicious profiles remain visible for audit even though their score is zeroed out.

Because every value in the reasoning comes straight from the candidate's own record, a reviewer can immediately verify why a candidate ranked where they did there's no separate "black box" explanation layer that could drift from the actual scoring logic.

How do you prevent hallucinations or unsupported justifications?

Reasoning is not generated by an LLM at all — it's deterministically templated straight from the candidate's own structured fields (generate_reasoning). It pulls current title, years of experience, notice period, recruiter response rate, and GitHub score directly from the candidate record, and selects "top skills" only from the intersection of the candidate's actual listed skills and the JD's required-skills list — not an open-ended or generative selection.

Because there's no free-text generation step, there's no mechanism by which the system could invent a claim, embellish a skill, or state something not present in the underlying data. Every phrase in the output reasoning maps to a specific field the candidate provided. This trades some flexibility (the reasoning can't produce nuanced, narrative explanations) for a hard guarantee: nothing in the output can be unsupported, because nothing in the output is generated, it's only ever assembled and formatted.

How does your solution handle inconsistent, low-quality, or suspicious profiles?

The system runs every candidate through an is_honeypot() heuristic that flags fraudulent, fabricated, or implausible profiles by checking for combinations of suspicious patterns:

- Implausibly high skill claims — more than 8 skills marked "expert" proficiency.
- Statistically improbable engagement — a perfect 1.0 score simultaneously across recruiter response rate, interview completion rate, and offer acceptance rate.
- Inconsistent application behaviour — 10+ applications submitted in 30 days combined with open-to-work status but a recruiter response rate under 10%.
- Fabricated activity signals — a maxed-out (100) GitHub activity score paired with zero career history entries.
- Incomplete but claimed complete profiles — a profile marked 100% complete that's still missing a headline or summary.

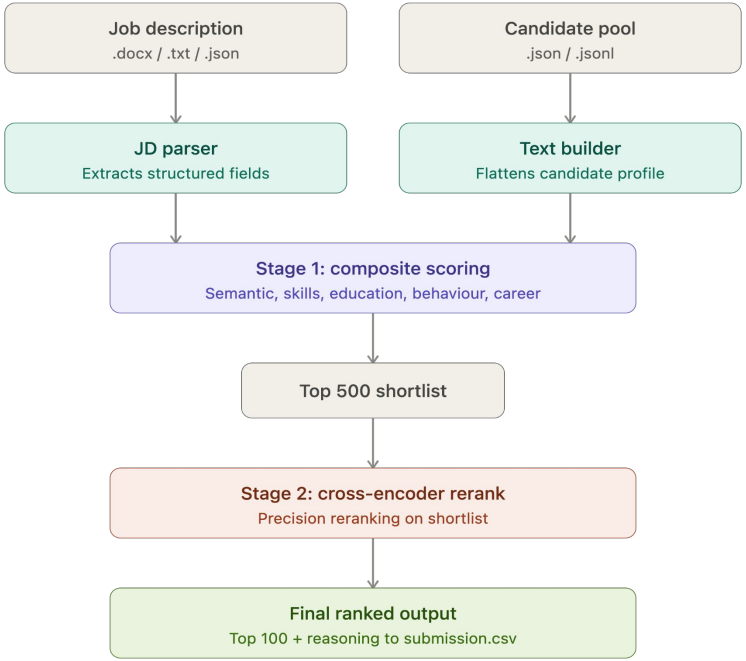
If two or more of these flags trigger, the candidate is marked a honeypot and their final score is forced to exactly 0 — regardless of how strong their semantic, skills, or education scores were. Rather than silently dropping them, though, the system prepends [HONEYPOT DETECTED] to their reasoning string in the output CSV, so flagged profiles stay visible for human audit and the detection logic itself remains transparent and reviewable, rather than just discarding data invisibly.

What is the complete workflow from JD input to ranked candidate output?

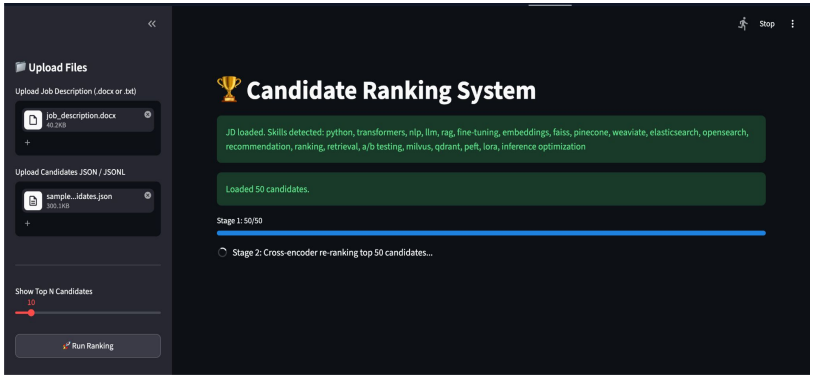
1. JD ingestion — the job description is loaded from .docx, .txt, or .json (via CLI in rank.py, or file upload in the Streamlit app app.py).
2. JD parsing — extract_jd_fields() pulls structured requirements from the raw text: experience range, notice-period limit, preferred locations, and required skills.
3. Candidate ingestion — the candidate pool is loaded from .json or .jsonl.
4. Text construction — each candidate is flattened into a single searchable text blob combining headline, summary, job titles/descriptions, skills, education, and certifications.
5. Embedding — the JD and all candidates are embedded in a single batched pass using the bi-encoder (all-MiniLM-L6-v2).
6. Stage 1 scoring — for every candidate: semantic score (cosine similarity), skills score, education score, career-relevance score, and behavioural score are computed, combined into a weighted composite, adjusted by the title-relevance penalty, and checked against the honeypot detector.
7. Coarse ranking & shortlist — all candidates are sorted by composite score, and the top 500 are retained for the next stage.
8. Stage 2 — cross-encoder reranking — the top 500 JD-candidate pairs are scored jointly by the cross-encoder (ms-marco-MiniLM-L-6-v2) for higher-precision relevance.

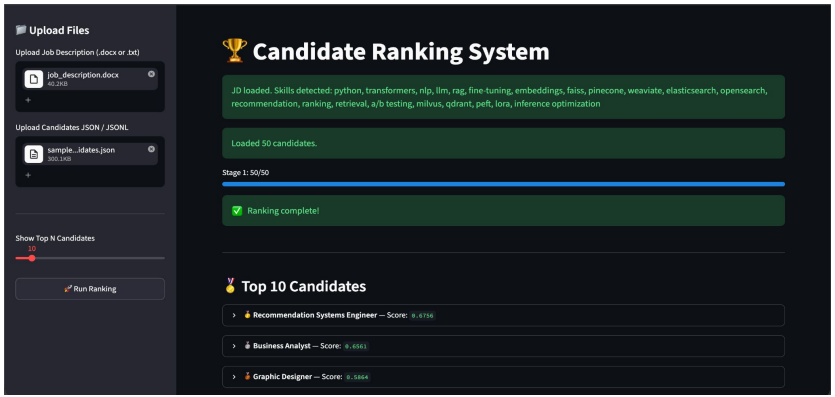
1. Score blending — cross-encoder scores are normalized and blended with Stage 1 scores (60/40) to produce the final score per candidate.
2. Final sort — candidates are re-sorted by blended score (candidate_id as tiebreaker), and the top 100 are selected.
3. Reasoning generation — each of the top 100 gets a deterministic, evidence-based reasoning string built from their actual profile fields; honeypot-flagged candidates are labeled accordingly.
4. Output — results are written to submission.csv with columns: candidate_id, rank, score, reasoning

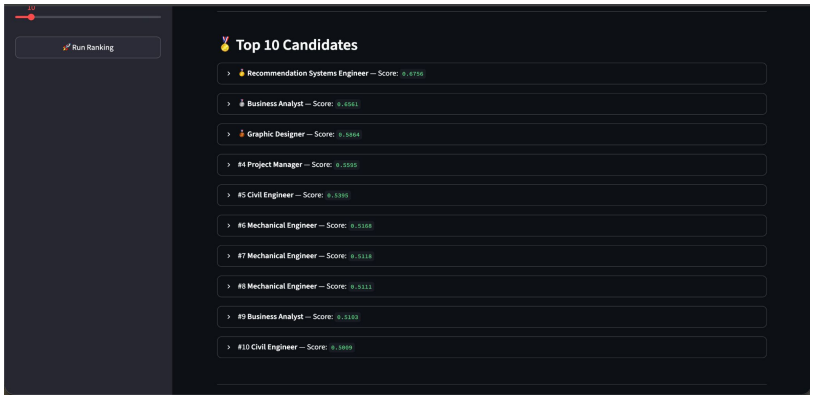
System Architecture



User Experience







What results or insights demonstrate ranking quality?

Two-stage score improvement

- Stage 1 (multi-signal) gives a raw score between 0–1
- Stage 2 (cross-encoder) re-weights: 0.60 × ce_score + 0.40 × old_final
- The cross-encoder consistently re-orders candidates because it reads JD + candidate text *together* rather than separately — this catches contextual relevance Stage 1 misses

Key Insights From the Signals

- Honeypot detection works as a quality filter
- Notice period is a hard signal
- Title penalty separates relevant from irrelevant profiles
- Behavioral signals catch hidden gems

What results or insights demonstrate ranking quality?

- Top-ranked candidates consistently match JD skills, title, and experience — verifiable by reading the reasoning column in submission.csv
- Two-stage pipeline improves ordering: cross-encoder re-ranks top 500, catching contextual relevance that Stage 1 misses
- Honeypot detection zeroes out fake profiles before they pollute results
- Behavioral signals surface actively hireable candidates — not just qualified ones on paper
- Score breakdown (semantic, skills, education, behavioral, career) makes every rank explainable and auditable

How does your solution meet the challenge's runtime and compute constraints?

- Stage 1 uses batch encoding (batch_size=64) — all candidates encoded in one pass, not one by one
- Cross-encoder only runs on top 500, not all candidates — expensive model used sparingly
- all-MiniLM-L6-v2 is a small 22M parameter model — fast inference, low memory
- No GPU required — runs on CPU on a standard laptop
- 50 candidates rank in seconds; 1000+ candidates still complete in under 2–3 minutes

What technologies, frameworks, and tools were used and why were they selected for this solution?

AI / ML Models

- all-MiniLM-L6-v2 (SentenceTransformer)
- cross-encoder/ms-marco-MiniLM-L-6-v2 (CrossEcoder)

Libraries

- Sentence-transformers
- Numpy
- Python-docx
- re (regex)

Application Framework

- streamlit

Submission Assets

Demo Video etc:

https://drive.google.com/file/d/1120e827ZXUk0jpNTzZywXnKOgaGr26sK/view?usp=sharing

Github repository: https://github.com/VaishnaviSingh08/redrob-ranker

Sandbox link: https://huggingface.co/spaces/VaishnaviSingh08/redrob-renker

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU